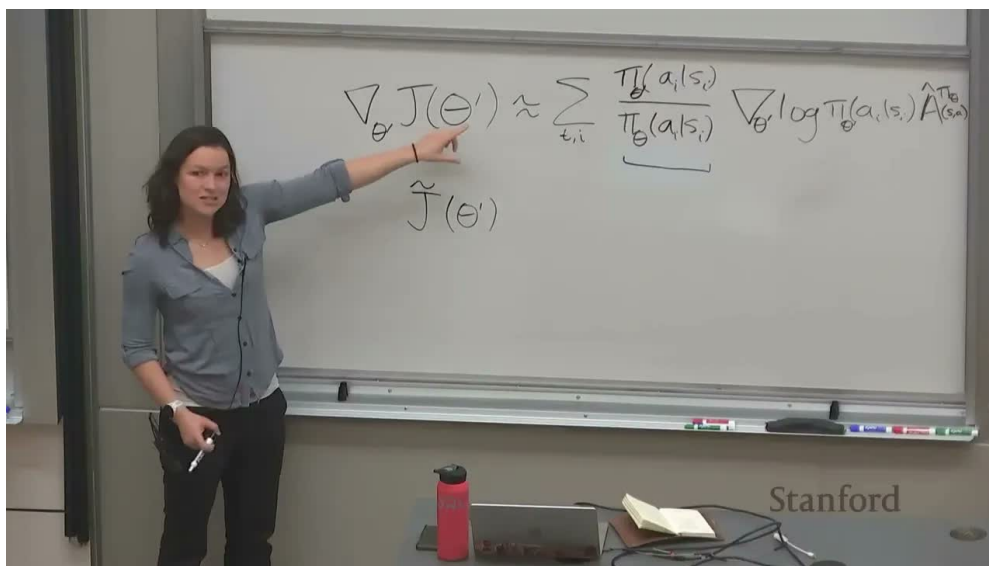


课程笔记

CS224R 第 5 讲: Off-Policy Actor-Critic

五道口纳什 & Codex

2025-04-16



视频作者/频道: Stanford CS224R

发布日期: 2025-04-16

视频时长: 01:09:17

视频链接: <https://www.youtube.com/watch?v=cRGKc-nAWho>

目录

1	为什么要更 off-policy: 从一次更新走向多次更新	2
1.1	importance ratio 的角色	2
1.2	为什么只靠 importance sampling 不够	2
1.3	本章小结	2
2	PPO: 把 actor-critic 做成稳定可用的工程算法	2
2.1	clipped surrogate objective	2
2.2	GAE: 用可调 horizon 估计 advantage	3
2.3	本章小结	3
3	从局部 off-policy 到 fully off-policy: replay buffer	4
3.1	问题一: 为什么不能继续直接拟合状态价值	4
3.2	问题二: 策略更新时应该信谁的动作	4
3.3	本章小结	5
4	SAC 与算法取舍: 数据效率、稳定性与应用场景	5
4.1	SAC 的高层图景	5
4.2	为什么 RL for language models 常提 PPO	6
4.3	本章小结	6
5	总结与延伸	6

1 为什么要更 off-policy: 从一次更新走向多次更新

上一讲的 actor-critic 已经把“估值”和“改策略”拆开，但整体仍偏 on-policy：数据来自当前策略，通常采一批数据只更新一次。这个代价在真实 RL 中非常高，因为与环境交互往往比网络反向传播昂贵得多。

本讲的第一步扩展，就是在同一批由旧策略 π_θ 收集的数据上，对新策略 $\pi_{\theta'}$ 做多次梯度更新。讲者提醒，这里的关键不在“多做几次优化”，而在于要正确处理新旧策略分布不一致的问题。

1.1 importance ratio 的角色

为了在旧数据上近似新策略目标，需要使用重要性权重

$$\rho_t = \frac{\pi_{\theta'}(a_t | s_t)}{\pi_\theta(a_t | s_t)}.$$

于是 surrogate objective 或等价梯度中，都会出现 $\rho_t \hat{A}_t$ 这一项。它的含义是：如果新策略更偏爱这个旧动作，就放大其影响；如果新策略更不偏爱它，就缩小其影响。

这个修正是 off-policy actor-critic 的第一道门槛。没有它，多次更新虽然表面上节省样本，但优化目标已经和真实策略目标脱节。

1.2 为什么只靠 importance sampling 不够

讲者也反复强调，importance sampling 不是银弹。若 $\pi_{\theta'}$ 与 π_θ 差得太远，则概率比会爆炸或塌缩，带来巨大方差。于是 practical RL 必须想办法约束“每一次策略更新不要走太远”。

1.3 本章小结

更 off-policy 的第一步不是 replay buffer，而是先允许一批数据支持多次更新；而一旦这样做，重要性采样与步长约束就成为必须处理的稳定性问题。

2 PPO: 把 actor-critic 做成稳定可用的工程算法

PPO 是本讲最重要的 practical algorithm。它本质上仍是 actor-critic，但在目标函数和优势估计上加入了若干技巧，使其在样本效率与训练稳定性之间取得实用平衡。

2.1 clipped surrogate objective

PPO 的核心目标写成

$$\tilde{J}(\theta') \approx \sum_{t,i} \min\left(\rho_{t,i} \hat{A}_{t,i}, \text{clip}(\rho_{t,i}, 1 - \epsilon, 1 + \epsilon) \hat{A}_{t,i}\right),$$

其中

$$\rho_{t,i} = \frac{\pi_{\theta'}(a_{t,i} | s_{t,i})}{\pi_\theta(a_{t,i} | s_{t,i})}.$$

与单纯 importance weighting 相比，PPO 在这里增加了 clip，硬性限制新旧策略比值的变化范围。这样一来，若策略更新想走得过远，目标函数会自动截断收益，防止优化器因为局部看似有利就大步偏航。

Proximal Policy Optimization (PPO)

1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_θ
2. Fit $\hat{V}_\phi^{\pi_\theta}$ to summed rewards in data
3. Evaluate $\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} w_n (\sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{t+n}))$
4. Update policy with M gradient steps on surrogate objective:

$$\tilde{J}(\theta') \approx \sum_{t,i} \min \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})} \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}), \text{clip} \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}, 1-\epsilon, 1+\epsilon \right) \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) \right)$$

Some example hyperparameters:

~2000 timesteps in batch of data

~10 epochs when updating policy clipping range $\epsilon=0.2$

($M \sim 300$ gradient steps with batch size 64)

~500 iterations $\rightarrow$ 1M total timesteps of experience

13

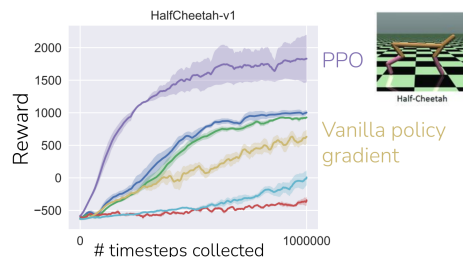


图 1: PPO 的训练流程、clip 目标与代表性超参数

图中同时给出了一个重要实践信号：PPO 虽然允许在同一批数据上做多次梯度更新，但它依旧不是 fully off-policy。它追求的是“稳定的局部多次利用”，而不是无限制复用历史经验。

2.2 GAE: 用可调 horizon 估计 advantage

PPO 还常与 generalized advantage estimation 配套。其基本思想是先得到一族不同步长的 advantage 估计

$$\hat{A}_n^\pi(s_t, a_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_\phi^\pi(s_t) + \gamma^n \hat{V}_\phi^\pi(s_{t+n}),$$

再做加权和

$$\hat{A}_{\text{GAE}}^\pi(s_t, a_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^\pi(s_t, a_t), \quad w_n \propto \lambda^{n-1}.$$

直觉上，较短 horizon 方差小、偏差大；较长 horizon 偏差小、方差大。GAE 用参数 λ 在两者之间平衡，使 advantage 更平滑、更适合作为 PPO 更新信号。

PPO 的本质

PPO 不是把 RL 变简单了，而是把“多次更新会失控”这件事直接写进目标函数里约束住，因此成为一个稳定、常用、对实现友好的 actor-critic 算法。

2.3 本章小结

PPO 通过 clip objective 和 GAE 两个关键组件，把“重要性采样修正”提升为真正好用的算法。它仍然基于 actor-critic，只是对目标构造和优势估计做了更强的工程化稳定处理。

3 从局部 off-policy 到 fully off-policy: replay buffer

PPO 只是在同一批数据上多更新几次，但本讲更进一步追问：能不能把所有过去的 trial-and-error 数据都留下来反复用？

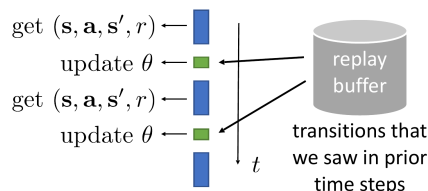
Off-Policy Actor-Critic Methods

Version 2: Replay buffers

actor-critic algorithm:

1. collect experience $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ } Add this to replay buffer
 2. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
 3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- Do this on minibatch sampled from all previous data

off-policy actor-critic



Slide adapted from Sergey Levine

15

图 2: replay buffer 版本的 off-policy actor-critic: 每次都从历史经验中采样 minibatch

这就是 replay buffer 思路。算法不再只依赖当前 batch，而是维护一个经验池 $\mathcal{R}$ ，其中每条 transition 形如 (s, a, s', r) 。训练时从 $\mathcal{R}$ 里抽样小批量，再执行参数更新。

它带来巨大样本效率优势，因为一段昂贵采样得到的数据，后面能被反复利用；但也立即引出两个理论和实现问题。

3.1 问题一：为什么不能继续直接拟合状态价值

若 replay buffer 中的数据来自大量旧策略，那么“状态 s 的未来回报”已经与旧策略动作分布纠缠在一起。直接拟合 $V^\pi(s)$ 时，标签到底对应哪个策略，变得含糊不清。

因此讲者提出更合适的思路：改学 $Q^\pi(s, a)$ 。因为数据集中本来就存了动作 a ，把动作当输入，就不必假装这些旧 transition 是由当前策略动作分布生成的。

3.2 问题二：策略更新时应该信谁的动作

即便有了 $Q(s, a)$ ，策略更新也不能直接拿 replay buffer 里旧动作照抄。因为旧策略的动作常常已经不是当前最优选择。策略梯度中的动作最好仍由当前 actor 采样，而 value estimator 则借助 replay buffer 提高样本利用率。

Version 2: Fixing the value function



1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
3. update $\hat{Q}_\phi^\pi$ using targets $y_i = r_i + \gamma \hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}'_i)$
4. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
5. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

higher variance, but okay b/c we're now using a lot more data (all data in buffer)

$$\text{sample } \mathbf{a}_i^\pi \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i) \quad \nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i^\pi|\mathbf{s}_i) \cdot \hat{Q}^\pi(\mathbf{s}_i, \mathbf{a}_i^\pi)$$

not from replay buffer $\mathcal{R}$!

20
Slide adapted from Sergey Levine

这张图非常关键。它说明 fully off-policy actor-critic 并不是“历史经验一把梭”，而是把 actor 与 critic 在数据使用上的角色重新分开：critic 更依赖 buffer，actor 更依赖当前策略采样或当前策略分布。

replay buffer 让 actor-critic 的样本效率大幅提升，但也迫使我们放弃一些只在 on-policy 下自然成立的简化公式。正是在这里，课程开始通向 SAC 一类 practical off-policy algorithm。

本讲后段用 SAC 作为代表 practical algorithm。SAC 的细节将在后续继续展开，但这里已经给出其定位：它是 fully off-policy、基于 replay buffer 的 actor-critic 代表，在数据效率上通常明显优于 PPO。

SAC 继续沿用 replay buffer 与 Q -function 学习框架，同时把最大熵目标引入策略优化。直观地说，它不仅让 actor 追求高 Q 值，还鼓励策略保留一定随机性，以改进探索与鲁棒性。

- PPO 更稳定、实现更顺手、在大规模 on-policy 采样条件下很实用。
- SAC 更省数据，但对超参数、目标构造和训练稳定性更敏感。

课程中的工程判断

如果你最在乎稳定性和易用性，PPO 往往是默认起点；如果你最在乎数据效率，尤其真实机器人交互昂贵，SAC 常常更有吸引力。

4.2 为什么 RL for language models 常提 PPO

讲者在结尾特别点出：PPO 是 language model RL 中常见的选择。原因并不神秘，而是因为 RLHF 这类流程天然更重视训练稳定性、约束更新幅度、避免策略突然崩坏。这和 PPO 的设计目标高度一致。

4.3 本章小结

本节给出 practical RL 的算法分工图谱：PPO 代表“稳定但不极致省样本”的 actor-critic，SAC 代表“更高样本效率但更难调”的 fully off-policy actor-critic。

5 总结与延伸

本讲真正完成的，不只是从 on-policy 到 off-policy 的字面变化，而是一次算法设计哲学的升级。上一讲的问题是“如何估值”；这一讲的问题变成“如何在估值的同时最大化复用数据，却不把训练搞崩”。

围绕这个目标，课程给出了两条主线。第一条主线是 PPO：接受数据仍然主要来自当前策略，但允许小心地多次复用同一批样本，并通过 clip 和 GAE 把不稳定性控制住。第二条主线是 replay buffer + Q -learning 式 critic：进一步拥抱历史数据，把数据效率推得更高，并最终导向 SAC。

这也解释了为什么 practical RL 没有单一最佳算法。不同任务在样本成本、动作空间、稳定性要求、工程复杂度上约束不同，导致 PPO 与 SAC 在工业和研究中长期并存。理解这点，比死记某一个目标函数更重要。